Bench — product requirements document

Version 1.0 · Status: draft for build · Owner: Rishivardhan J

1. Overview

Bench is a two-sided talent marketplace. Freelancers sign up, build a profile with their skills, resume and portfolio, and wait to be discovered. Clients (anyone who needs work done) sign in separately, search freelancers by skills, and connect with them directly inside the app.

One-liner: Find the right freelancer by describing the job, then talk to them without leaving Bench.

What changed from earlier plans: Bench is no longer a search tool over someone else's data, and it is not a dataset or model-training project. Every freelancer on Bench is a real person who created their own profile. AI is used quietly in the background (understanding a typed brief, reading a resume) and is never presented as a feature or a dashboard.

2. Goals and non-goals

Goals

- A client can go from a plain-language description to a shortlist and a first message in under three minutes.
- A freelancer can build a complete, credible profile in under ten minutes, starting from an uploaded resume.
- Trust is built into the product: moderated profiles, verified badges, and reviews that only come from real engagements.
- The interface looks like a finished product within the first five seconds: dense, calm, monochrome.

Non-goals for v1

- Payments, escrow and invoicing.
- Contracts, milestones and time tracking.
- Video calls.
- Native mobile apps (the web app must be fully responsive).
- Visible AI features, chat-with-an-agent interfaces, or model metrics.

3. Users

Persona	Who	What they need
Client	Founder, small team, hiring manager, student with a project	Find a qualified person fast, judge them quickly, contact them, keep a shortlist
Freelancer	Designer, developer, writer, analyst, video editor	Be found for the right work, control who contacts them, look credible
Admin	The Bench team (initially one person)	Approve profiles, handle reports, keep spam out

4. Account model

There are two separate account types plus an internal admin role.

- **Freelancer account** and **client account** have separate sign-up flows, separate dashboards and separate data. One email can hold only one account type (a Firebase Auth constraint); the error message must say this clearly.
- Sign-in is email and password only. Google sign-in was removed and stays out of v1.

- A user's role is stored on their user record and enforced in routing and in security rules. A freelancer can never open client pages and the reverse.
- Search stays open to signed-out visitors. Saving, messaging and applying require an account.
- Account deletion is self-serve and removes profile, messages and shortlists.

5. Core flows

Freelancer: onboarding

1. Choose "For freelancers" and create an account.
2. Upload a resume (PDF). Bench reads it in the background and pre-fills name, role, skills and experience. The freelancer reviews and edits every field.
3. Add rate, availability, timezone, portfolio links and an avatar.
4. Submit. The profile enters review ("Pending review").
5. After approval the profile is live and discoverable. Any later edit to core fields returns it to review.

Client: find and connect

1. Land on search. Type a sentence such as "senior React developer, Firebase, $80–120/hr, start in two weeks", or use skill filters.
2. Bench quietly extracts skills, budget and timeline, applies them as filters, and ranks results. Each result carries one line explaining the match.
3. Open a profile, add to a shortlist, or press Connect.
4. Connect opens a short request form: a message and an optional project brief attached.
5. The freelancer accepts or declines. On accept, a private message thread opens for both.

Client: post a project (new)

1. Create a project with title, description, required skills, budget range and timeline.
2. Matching freelancers are notified; others can find it in a project board.
3. Freelancers send a short proposal. The client reviews proposals, shortlists, and opens a thread with chosen people.

6. Feature requirements

Priority: **P0** must ship in v1, **P1** v1.1 soon after, **P2** later.

6.1 Freelancer side

ID	Feature	Priority	Notes
F1	Profile builder	P0	Name, headline, bio, skills (tag input), rate range and currency, availability, timezone, portfolio URL, languages
F2	Resume upload and auto-fill	P0	PDF upload; skills and experience are extracted server-side and offered as editable suggestions, never saved without confirmation
F3	Avatar upload	P0	Image only, size-limited, initials fallback
F4	Portfolio gallery	P1	Up to six items with title, image, link and one-line description
F5	Experience and education	P1	Structured entries, shown on the profile
F6	Profile completeness meter	P1	Plain percentage with a checklist; profiles above 80% rank slightly higher
F7	Availability status	P0	Available now, this week, busy, offline; optional "back on" date

ID	Feature	Priority	Notes
F8	Connection inbox	P0	Incoming requests; accept, decline, or block. Freelancers control who can message them
F9	Project board and proposals	P1	Browse open projects that match their skills and send a short proposal
F10	Profile analytics	P1	Views, shortlist adds, requests received, simple weekly trend
F11	Profile status page	P0	Pending review, live, or changes requested with a reason
F12	Public profile link	P1	Shareable URL with clean preview metadata
F13	Pause profile	P1	Hide from search without deleting

6.2 Client side

ID	Feature	Priority	Notes
C1	Skill search and filters	P0	Multi-select skills, rate range, availability, minimum rating, timezone overlap, language
C2	Sentence search	P0	Type the job in plain words; keywords are extracted and applied as visible filter chips. Works without the user seeing any AI label
C3	Ranked results with match line	P0	One sentence per result on why it fits. Sort by relevance, rate, rating, response time
C4	Freelancer profile view	P0	Full bio, skills, portfolio, experience, reviews, response time, timezone and local time
C5	Shortlists	P0	Multiple named lists, notes on each freelancer
C6	Connect request	P0	Message plus optional brief; limited sends per day to prevent spam
C7	Post a project	P1	Title, description, skills, budget, timeline; edit and close
C8	Proposals inbox	P1	Compare proposals side by side, shortlist, message
C9	Compare mode	P1	Two or three freelancers side by side on rate, skills, rating, availability
C10	Saved searches with alerts	P1	Email or in-app alert when new matching freelancers go live
C11	Hire tracking	P0	Mark a freelancer as hired for a project; this unlocks reviews
C12	Leave a review	P0	Only after a hire is marked and a minimum conversation exists
C13	Recently viewed	P2	Quick return to profiles

6.3 Shared features

ID	Feature	Priority	Notes
S1	In-app messaging	P0	Text threads, attachments (files and images, size-limited), read state, calm and minimal
S2	Notifications	P0	In-app bell plus email for new request, accepted request, new message, proposal received
S3	Notification preferences	P1	Per type, email on or off
S4	Report and block	P0	Report a profile or a message; block a user from contacting you
S5	Command palette	P1	Cmd+K to jump to search, shortlists, inbox, settings
S6	Keyboard navigation in results	P2	Arrow keys through rows, Enter to open, S to shortlist
S7	Empty, loading and error states	P0	Skeletons that match final layout; actionable empty states; retry on error

ID	Feature	Priority	Notes
S8	Accessibility	P0	Visible focus, labelled icon buttons, live regions for async status, skip link

6.4 Trust and safety

ID	Feature	Priority	Notes
T1	Profile moderation	P0	New and edited profiles start as pending; approved by an admin
T2	Verified badge	P1	Granted manually after checking portfolio or LinkedIn and email; one meaning only
T3	Verified reviews	P0	Reviews require a marked hire; one review per engagement; freelancer may reply once
T4	Rate limits	P0	Connect requests, messages to new contacts, and sign-ups are limited per account and IP
T5	Off-platform guard	P1	Warn when a message includes a phone number or external payment link in the first exchange
T6	Admin panel	P1	Review queue, reports, user lookup, approve or request changes with a reason. Until built, use the Firebase console

6.5 How AI is used (invisible)

- **Brief understanding:** turn a typed sentence into skills, budget band, timeline and seniority. Output appears only as filter chips the user can change.
- **Match line:** one sentence per result built from the actual overlap (skills, rate fit, availability).
- **Resume reading:** extract skills and experience to pre-fill the profile, always editable.
- **Review summary:** two or three sentences condensing reviews on a profile, shown under a plain "What people say" heading, with the raw reviews one click away.
- All calls run server-side with a Gemini key kept out of the browser. Brief text is treated as untrusted input, length-capped and rate limited. No AI badges, no confidence scores, no chat with an assistant.

7. Pages and navigation

Public: search home, freelancer profile, for-freelancers landing, privacy, terms, 404.

Client area: shortlists, projects, proposals, messages, notifications, settings.

Freelancer area: profile editor, status, requests, messages, project board, analytics, settings.

Admin: review queue, reports.

Top bar holds the wordmark, Search, Messages, notifications bell and an account menu. The footer has For freelancers, Privacy and Terms.

8. Design system (unchanged palette)

The visual language stays exactly as built. New screens reuse these tokens and components and add nothing new.

Token	Value	Use
bg	#000000	Page background
surface	#0A0A0A	Cards and rows

Token	Value	Use
surface-2	#111111	Hover
surface-3	#171717	Avatars, chips, dropdowns
border	#1F1F1F	Hairlines
border-strong	#2E2E2E	Hover and emphasis
text	#FFFFFF	Primary text
text-dim	#A1A1A1	Secondary text
text-mute	#666666	Metadata, placeholders
accent	#3291FF	The only accent, two to three uses per screen
accent-dim	#0C1B2E	Accent badge background
accent-text	#52A9FF	Accent text on dark

Rules

- Inter for interface text (weights 400 to 700), JetBrains Mono for numbers: rates, ratings, counts, response times. All numbers use tabular figures and right-align in columns.
- Spacing in multiples of 4px. Radii 6, 8 and 10 to 12px by tier.
- No gradients, no shadows except a neutral focus ring, no emoji, no extra colours. Status dots are white, grey or hollow, never green, red or amber.
- Primary button is white fill with black text; secondary is transparent with a strong border. One primary button per screen.
- Sentence case everywhere. Motion is limited to 120ms row hover, a 150 to 200ms chip fade-in, and a 0.97 button press.
- Tables share one grid template between header and rows; long text truncates with an ellipsis; no layout shift when data loads.
- Messaging follows the same system: dark surface, surface-2 for incoming bubbles, white fill for outgoing, no coloured read receipts (use text-mute check marks).
- Unread counts and notification dots use the accent, and count toward the two-to-three accent budget.

9. Data model (Firestore)

```
users/{uid}                    role: client | freelancer | admin, email, createdAt
freelancerProfiles/{uid}       name, headline, bio, skills[], rateMin, rateMax, currency,
                               availability, timezone, portfolioUrl, avatarUrl, languages[],
                               experience[], education[], portfolio[], resumePath,
                               approved, verified, paused, completeness, ratingAvg,
                               reviewCount, avgResponseMinutes, createdAt, updatedAt
clientProfiles/{uid}           name, company, avatarUrl
shortlists/{clientUid}/lists/{listId}   name, items[{freelancerId, note, addedAt}]
projects/{projectId}           clientUid, title, description, skills[], budgetMin,
                               budgetMax, timeline, status: open | closed, createdAt
proposals/{proposalId}         projectId, freelancerUid, message, rate, status
connections/{connectionId}     clientUid, freelancerUid, projectId?, status: pending |
                               accepted | declined | blocked, firstMessage, createdAt
threads/{threadId}             participants[], connectionId, lastMessageAt, unread{uid: n}
threads/{threadId}/messages/{id}  senderUid, text, attachments[], createdAt, readBy[]
hires/{hireId}                 clientUid, freelancerUid, projectId?, markedAt
reviews/{reviewId}             hireId, clientUid, freelancerUid, rating, text, reply?
notifications/{uid}/items/{id} type, refId, read, createdAt
savedSearches/{id}             clientUid, query, filters, alertEnabled
reports/{id}                   reporterUid, targetType, targetId, reason, status
```

Rules of thumb for security

- Public read of a freelancer profile only when approved is true and paused is false; owners always read their own.
- Freelancers write only their own profile and cannot set approved, verified, ratingAvg or reviewCount. Those are set by admin or server functions.
- Message threads are readable and writable only by the two participants. A thread exists only after a connection is accepted.
- A review can be created only if a matching hire exists and none was written yet.
- Resumes live in private storage, readable only by the owner and admin.

10. Technical approach

Layer	Choice
Frontend	React, TypeScript, Vite, Tailwind with the tokens above, TanStack Query, Framer Motion (sparingly), Lucide icons
Auth	Firebase Auth, email and password
Data	Firestore with real-time listeners for messages and notifications
Files	Firebase Storage for avatars, resumes, attachments, portfolio images
Server logic	Firebase Functions: brief understanding, resume reading, review summary, notification fan-out, rate limiting, rating recalculation
AI	Gemini API from Functions only
Search	Start with Firestore queries and client-side ranking over approved profiles; move to a hosted search service when the pool passes a few thousand profiles
Email	A transactional email provider triggered from Functions
Monitoring	Error tracking on frontend and Functions
Hosting	Vercel for the frontend, Firebase for backend

Freelancer data access goes through a single provider interface so the search page does not care where profiles come from.

11. Search and ranking behaviour

- Only approved, unpaused profiles are searchable.
- Score combines skill overlap, text match against headline and bio, rate fit, availability, completeness and (when present) rating. Profiles with no rating sort after rated ones for rating sort, and show "New" instead of a number.
- Response time is computed from real request replies; until a freelancer has a few replies it is hidden, not shown as zero.
- Typing in the search box updates results after a short debounce; no search button.
- Every filter the system extracted from a sentence is shown as a removable chip.

12. Success metrics

Area	Metric	Early target
Supply	Approved freelancer profiles	50 before public launch
Activation	Freelancers who finish a profile within 24 hours of sign-up	60%
Demand	Clients who send at least one connect request in first session	25%

Area	Metric	Early target
Liquidity	Connect requests accepted	50%
Speed	Median time from first search to first request	under 3 minutes
Quality	Reviews left per marked hire	40%
Trust	Reports per 1,000 profile views	under 2

13. Release plan

v1 (launchable core): Freelancer accounts and profile builder with resume auto-fill, moderation, client accounts, skill and sentence search, profile view, shortlists, connect requests, messaging, notifications, hire marking and reviews, report and block, rate limits, legal pages, accessibility and loading states.

v1.1: Post a project and proposals, portfolio gallery, profile analytics, compare mode, saved searches with alerts, verified badge workflow, admin panel, command palette, public profile links.

v2: Payments and escrow, milestones, calendar scheduling, team accounts for clients, native apps, multi-language interface.

14. Risks and open questions

Risk or question	Plan
Cold start: no freelancers, no clients	Personally recruit the first 50 freelancers; launch with a waitlist; seed niche by skill category
Spam and off-platform leakage	Request limits, freelancer-controlled inbox, off-platform warning, block and report
Fake or low-quality profiles	Manual approval, edit-resets-approval rule, verified badge with a single clear meaning
Reviews with no payments to anchor them	Reviews unlock only after a marked hire plus a real conversation; revisit when payments exist
Moderation workload	Start in Firebase console; build admin panel in v1.1
Privacy of resumes and messages	Private storage, participant-only message rules, deletion on request, updated privacy policy before launch
AI cost and abuse	Server-side only, length caps, per-user rate limits, cache resume and summary results
Same email as both roles	Not supported in v1; clear error message; revisit with multi-role accounts later
Legal text	Current privacy and terms pages are a draft and need legal review before a public launch

Open decisions

1. Should clients see a freelancer's contact details after acceptance, or only chat inside Bench? Recommended: chat only in v1.
2. Should the first connect request be free of limits for new clients? Recommended: three per day, raised after a client's first accepted request.
3. Should freelancers be allowed to message first on projects? Recommended: yes, through proposals only.
4. Naming and domain: Bench is a working name.

15. Acceptance for v1

- A new freelancer can finish a profile from a resume, submit it, and see its status.
- An admin approval makes the profile appear in search within seconds.
- A client can search by skill and by sentence, open a profile, shortlist it, send a connect request, and chat after acceptance.
- A hire can be marked and a review left, and it appears with the correct rating on the profile.
- A freelancer cannot see client-only pages, and the reverse. One user cannot read another user's threads, shortlists or private files.
- Every data view has loading, empty and error states, and passes the keyboard and contrast checks.
- Visual audit passes: tokens only, 4px spacing, accent used no more than three times per screen, tables aligned to one grid.